

Pseudo Code

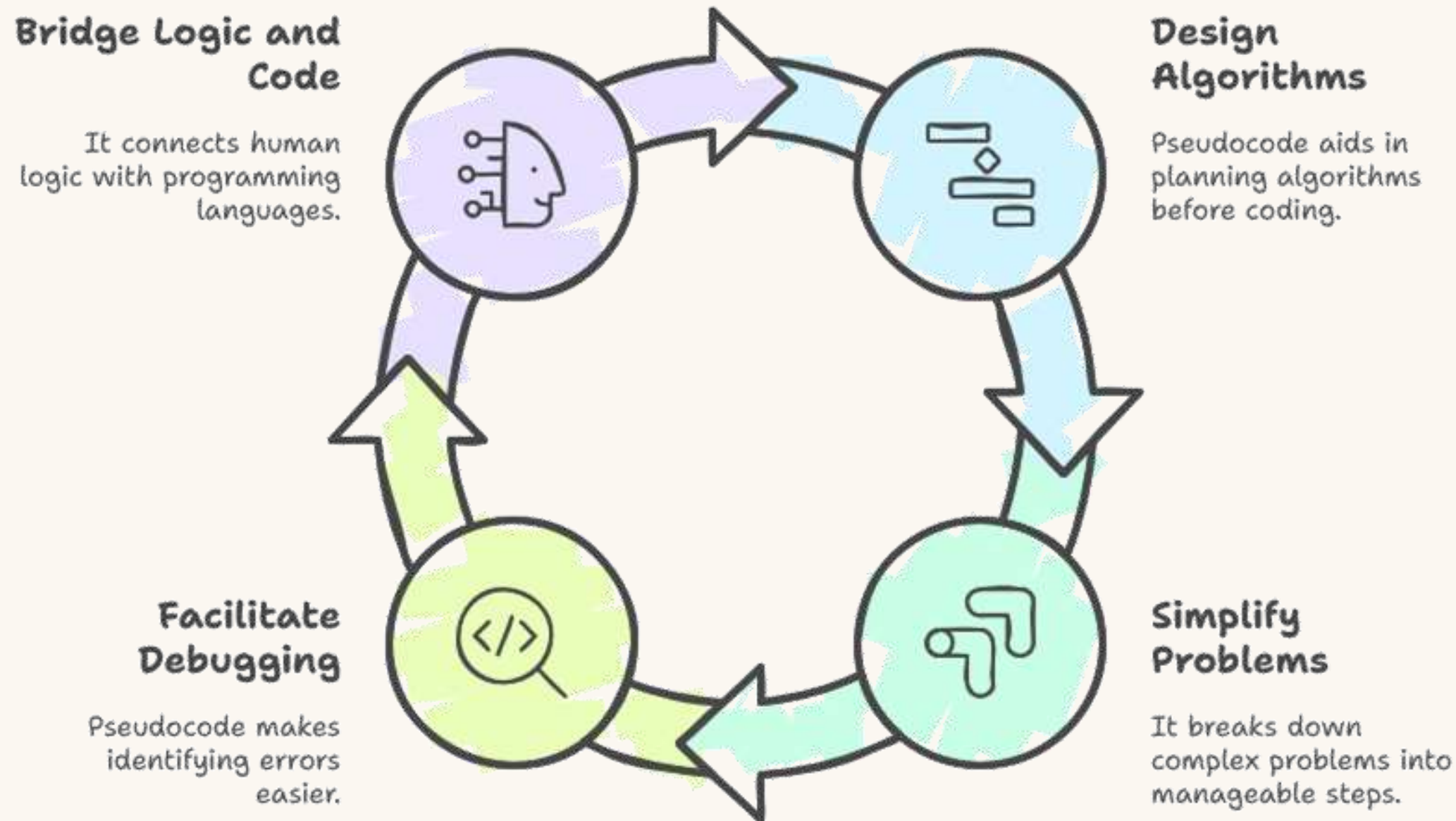
Programming Concepts

Contents

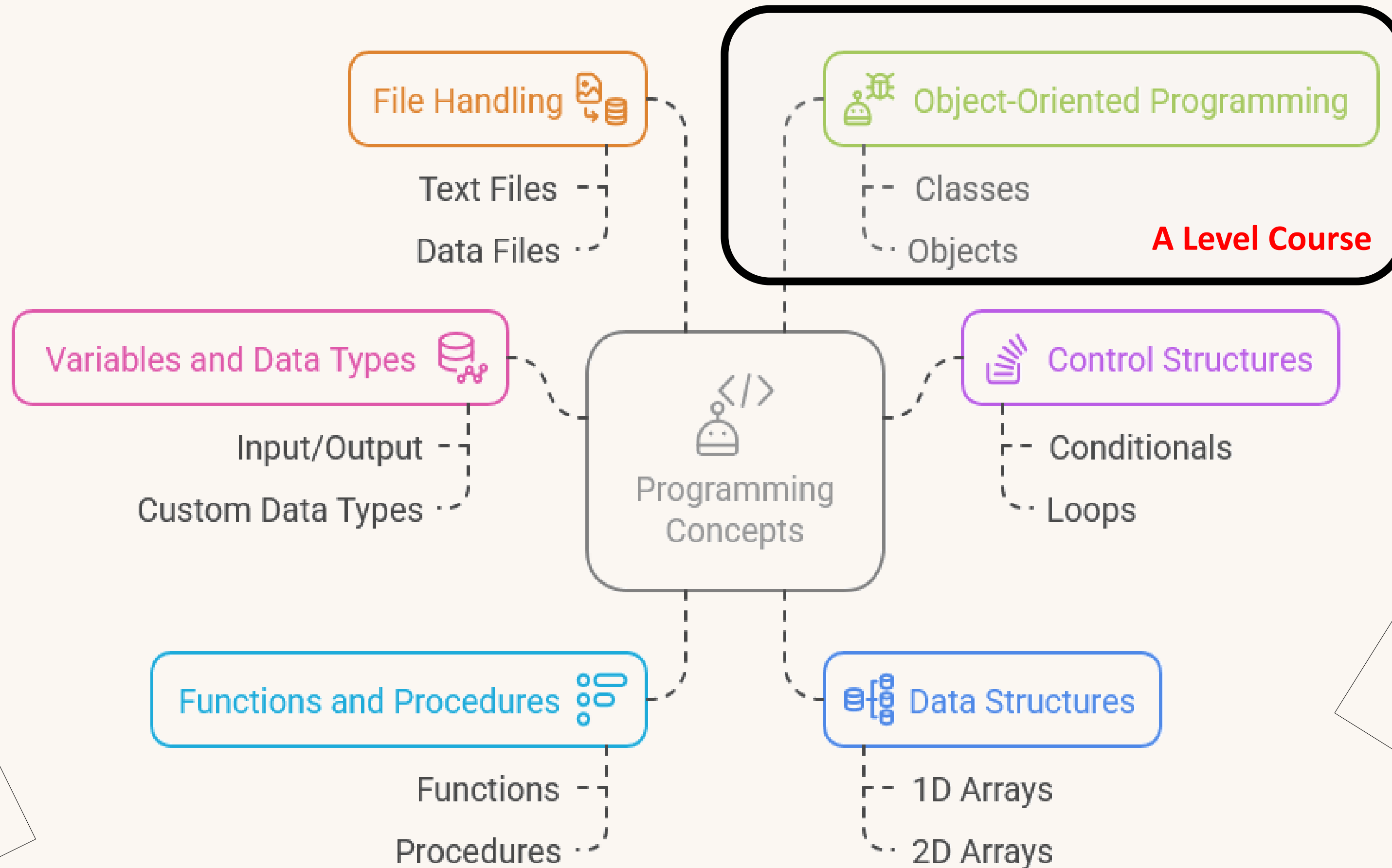
1. Programming Concepts & Pseudo Code in detail
2. Examples
3. Past Papers
4. Questions & Answers
5. Practice Session
6. Quires W.R.T other CS Topics

Introduction

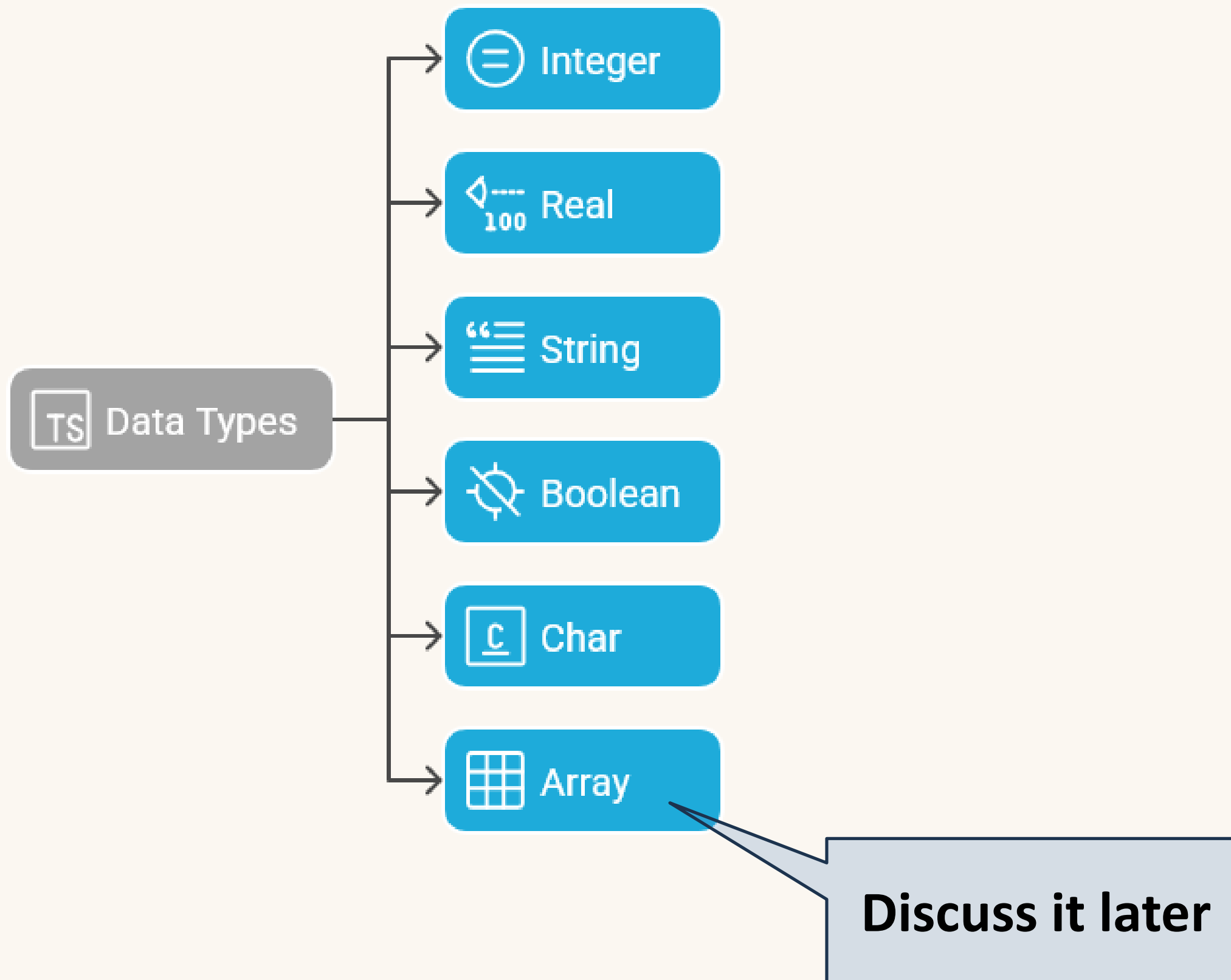
Pseudocode is a method of designing algorithms using a structured yet readable format that is independent of any programming language. It allows programmers and developers to plan their logic before implementing it in an actual programming language.



Programming Concepts and Data Structures



Variable and Declarations – Data Types



DECLARE age : INTEGER
DECLARE name : STRING
DECLARE temperature : REAL
DECLARE isPassed : BOOLEAN
DECLARE grade : CHAR

age ← 46
name ← "Mustafa Khan"
temperature ← 36.5
isPassed ← TRUE
grade ← 'A'

Good Practices for Variable Declaration

Use Descriptive and Valid Variable Names

X ← 0

Bad

TotalMarks ← 0

Good

Declare Variables Before Use (Optional but Good Practice)

IGCSE doesn't require formal declaration, but it's good to show **intended data types**.

Be Consistent With Naming Style

- Use **camelCase** or **lower_case_with_underscores** (choose one and stick with it).
- Avoid spaces or confusing abbreviations.

- studentAge
- totalPrice
- isValid

Initialize Variables If Needed

DECLARE total : INTEGER

total ← 0

Variable and Declarations (Example)

DECLARE Name : STRING

DECLARE Age : INTEGER

OUTPUT "Enter your name: "

INPUT Name

OUTPUT "Enter your age: "

INPUT Age

OUTPUT "Hello, ", Name, "! You are ", Age, " years old."

<https://cs.coursemo.com/igpc/>
<https://pseudocode.pro>

Variable and Declarations

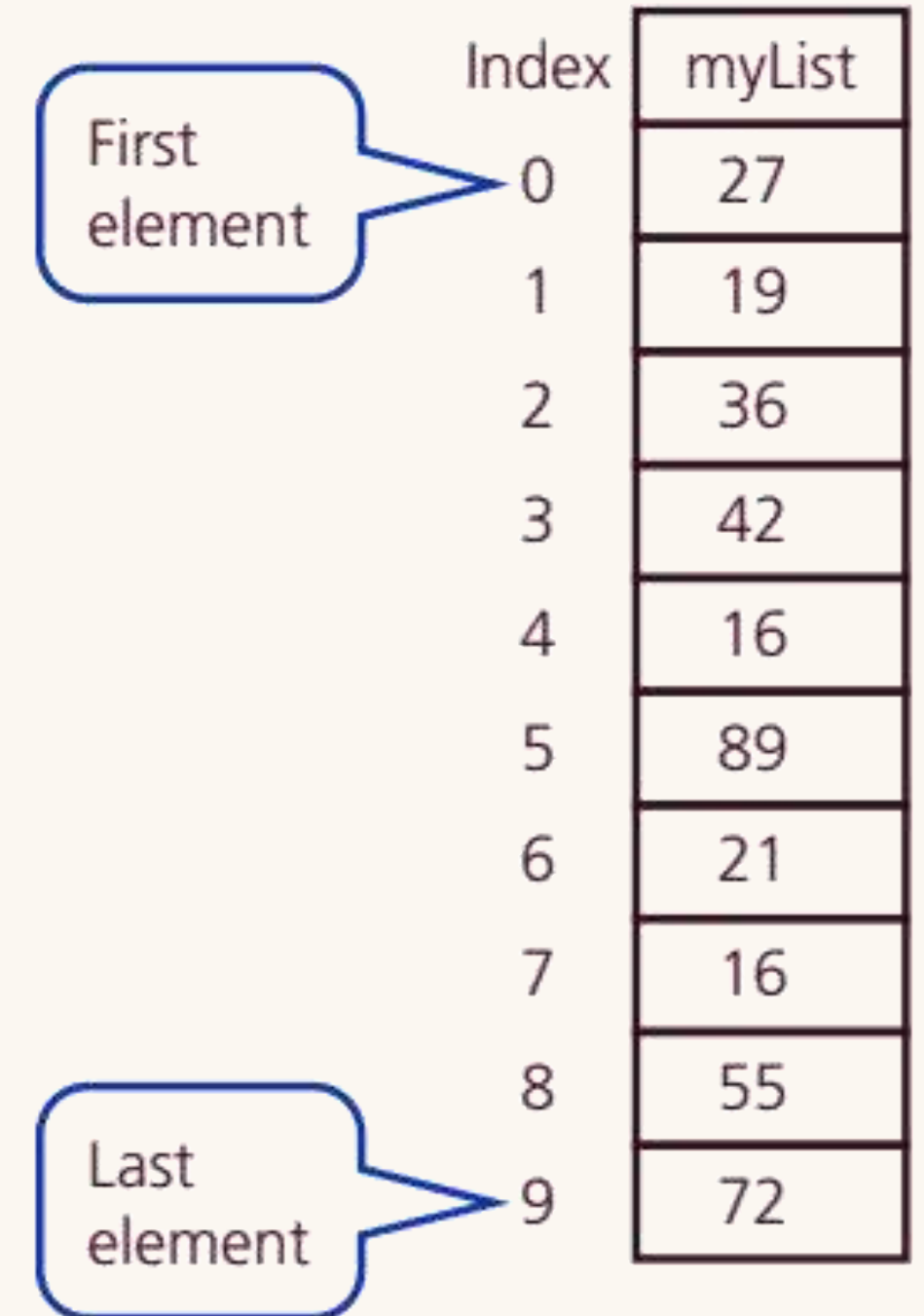
1D Arrays [.....]

When a one-dimensional array is declared in pseudocode:

- the name of the array
- the first index value
- the last index value
- the data type

```
DECLARE MyList : ARRAY[0:9] OF INTEGER
```

```
MyList[5] ← 15
```



Index	myList
0	27
1	19
2	36
3	42
4	16
5	89
6	21
7	16
8	55
9	72

First element

Last element

Variable and Declarations (Example)

ID Arrays [.....]

```
DECLARE MyList : ARRAY[0:4] OF INTEGER
```

```
OUTPUT "Enter these 5 values in order 1 2 3 4 5 “
```

```
MyList[0] ← 1
```

```
MyList[1] ← 2
```

```
MyList[2] ← 3
```

```
MyList[3] ← 4
```

```
MyList[4] ← 5
```

```
OUTPUT "Value 1 is : ", MyList[0]
```

```
OUTPUT "Value 2 is : ", MyList[1]
```

```
OUTPUT "Value 3 is : ", MyList[2]
```

```
OUTPUT "Value 4 is : ", MyList[3]
```

```
OUTPUT "Value 5 is : ", MyList[4]
```

Variable and Declarations

2D Arrays [.....][.....]

When a Two-dimensional array is declared in pseudocode:

- the name of the array
- the first index value for Row
- the last index value for Row
- the first index value for Columns
- the last index value for Columns
- the data type

```
DECLARE MyList : ARRAY[0:9,0:2] OF INTEGER
```

```
MyList[5,1] ← 25
```

MyTable

27	31	17
19	67	48
36	98	29
42	22	95
16	35	61
89	46	47
21	71	28
16	23	13
55	11	77
34	76	21

First element

rows

Last element

columns

Variable and Declarations (Example)

2D Arrays [.....][.....]

```
DECLARE MyList : ARRAY[0:2,0:2] OF INTEGER
```

```
OUTPUT "Enter these 3 values in order 1 2 3 for Rows & Columns “
```

```
MyList[0,0] ← 1
```

```
MyList[0,1] ← 1
```

```
MyList[0,2] ← 1
```

```
MyList[1,0] ← 2
```

```
MyList[1,1] ← 2
```

```
MyList[1,2] ← 2
```

```
MyList[2,0] ← 3
```

```
MyList[2,1] ← 3
```

```
MyList[2,2] ← 3
```

```
OUTPUT "Value 1 is : ", MyList[0,0]
```

```
OUTPUT "Value 2 is : ", MyList[0,1]
```

```
OUTPUT "Value 3 is : ", MyList[0,2]
```

```
OUTPUT "Value 4 is : ", MyList[1,0]
```

```
OUTPUT "Value 5 is : ", MyList[1,1]
```

```
OUTPUT "Value 6 is : ", MyList[1,2]
```

```
OUTPUT "Value 7 is : ", MyList[2,0]
```

```
OUTPUT "Value 8 is : ", MyList[2,1]
```

```
OUTPUT "Value 9 is : ", MyList[2,2]
```

Variable and Declarations (Example)

2D Arrays [.....] [.....]

```
DECLARE MyTable : ARRAY[0:2,0:2] OF INTEGER
```

```
DECLARE ColumnCounter : INTEGER
```

```
DECLARE RowCounter : INTEGER
```

```
DECLARE Value : INTEGER
```

```
OUTPUT "Enter these 9 values in order 1 2 3 4 5 6 7 8 9 for rows"
```

```
FOR ColumnCounter <- 0 TO 2
```

```
    FOR RowCounter <- 0 TO 2
```

```
        OUTPUT "Enter next value "
```

```
        INPUT Value
```

```
        MyTable[RowCounter, ColumnCounter] <- Value
```

```
    NEXT RowCounter
```

```
NEXT ColumnCounter
```

```
FOR ColumnCounter <- 0 TO 2
```

```
    FOR RowCounter <- 0 TO 2
```

```
        OUTPUT MyTable[RowCounter,ColumnCounter]
```

```
    NEXT RowCounter
```

```
NEXT ColumnCounter
```

**Count
Controlled Loop**

If Statements

**Pre-condition
Loops**

**If Else
Statements**

**Post Conditions
Loops**

Cass Statements

Loops & Conditionals

IF Statement

```
IF condition THEN
```

```
    // Code to execute if condition is true
```

```
ENDIF
```

```
IF height > 6
```

```
    THEN
```

```
        OUTOUT “ You are a tall person”
```

```
ENDIF
```

IF & ELSE Statement

```
IF condition THEN
```

```
    // Code if condition is true
```

```
ELSE
```

```
    // Code if condition is false
```

```
ENDIF
```

```
IF height > 6
```

```
    THEN
```

```
        OUTPUT " You are a tall person"
```

```
    ELSE
```

```
        OUTPUT " You have shorter Height"
```

```
ENDIF
```

IF, ELSEIF & ELSE Statement

```
IF condition1 THEN
    // Code if condition1 is true
ELSEIF condition2 THEN
    // Code if condition2 is true
ELSE
    // Code if neither condition1 nor condition2 is true
ENDIF
```

```
IF Height > 6
    THEN
        OUTPUT " You are a tall person"
    ELSEIF Height < 5
        OUTPUT " You have shorter Height"
    ELSE
        OUTPUT " You have Average Height"
ENDIF
```


IF, ELSEIF & ELSE Statement (Example)

```
DECLARE Marks : INTEGER
```

```
OUTPUT "Enter your marks: "
```

```
INPUT Marks
```

```
    IF Marks >= 80 THEN
```

```
        OUTPUT "Grade: A"
```

```
    ELSE IF Marks >= 60 THEN
```

```
        OUTPUT "Grade: B"
```

```
    ELSE IF Marks >= 40 THEN
```

```
        OUTPUT "Grade: C"
```

```
    ELSE OUTPUT "Grade: F"
```

```
ENDIF
```

IF & ELSE Statement with Double Conditions (Examples)

```
DECLARE Age : INTEGER
```

```
DECLARE HASTICKET : BOOLEAN
```

```
OUTPUT "Enter your age: "
```

```
INPUT Age
```

```
OUTPUT "Do you have a ticket? (TRUE/FALSE): "
```

```
INPUT HASTICKET
```

```
IF Age >= 18 AND HASTICKET THEN
```

```
    OUTPUT "You can enter the event."
```

```
ELSE
```

```
    OUTPUT "You cannot enter the event."
```

```
ENDIF
```

With **AND** both
Conditions must be true

IF & ELSE Statement with Double Conditions (Examples)

```
DECLARE HasDiscountCard : BOOLEAN
DECLARE IsStudent : BOOLEAN

OUTPUT "Are you a student? (TRUE/FALSE): "
INPUT IsStudent
OUTPUT "Do you have a discount card? (TRUE/FALSE): "
INPUT HasDiscountCard

IF IsStudent OR HasDiscountCard THEN
    OUTPUT "You get a discount!"
ELSE
    OUTPUT "No discount available."
ENDIF
```

With **OR** – At least one
conditions must be true

IF & ELSE Statement with Double Conditions (Example)

```
DECLARE Temperature : INTEGER
DECLARE Israining : BOOLEAN
DECLARE Hasumbrella : BOOLEAN

OUTPUT "Enter the temperature: "
INPUT Temperature
OUTPUT "Is it raining? (TRUE/FALSE): "
INPUT Israining
OUTPUT "Do you have an umbrella? (TRUE/FALSE): "
INPUT Hasumbrella

IF (Temperature < 10 AND Israining) OR (Hasumbrella = FALSE)
THEN
    OUTPUT "Wear a jacket and carry an umbrella!"
ELSE
    OUTPUT "You're good to go!"
ENDIF
```

Using both **AND & OR**
together

IF & ELSE Statement with Double Conditions (Example)

```
DECLARE Username : STRING
DECLARE Password : STRING

OUTPUT "Enter username: "
INPUT Username
OUTPUT "Enter password: "
INPUT Password

IF Username = "admin" AND Password = "1234" THEN
    OUTPUT "Login successful!"
ELSE
    OUTPUT "Invalid username or password."
ENDIF
```

Checking Multiple
Conditions

CASE Statement

CASE OF variable

value1:

// Code block for value1

value2:

// Code block for value2

...

OTHERWISE:

// Code block if no match is found

ENDCASE

DECLARE Choice : INTEGER

OUTPUT "Enter your choice (1-3): "

INPUT Choice

CASE OF Choice

1:

OUTPUT "You selected Option 1 - View Balance"

2:

OUTPUT "You selected Option 2 - Deposit Money"

3:

OUTPUT "You selected Option 3 - Withdraw Money"

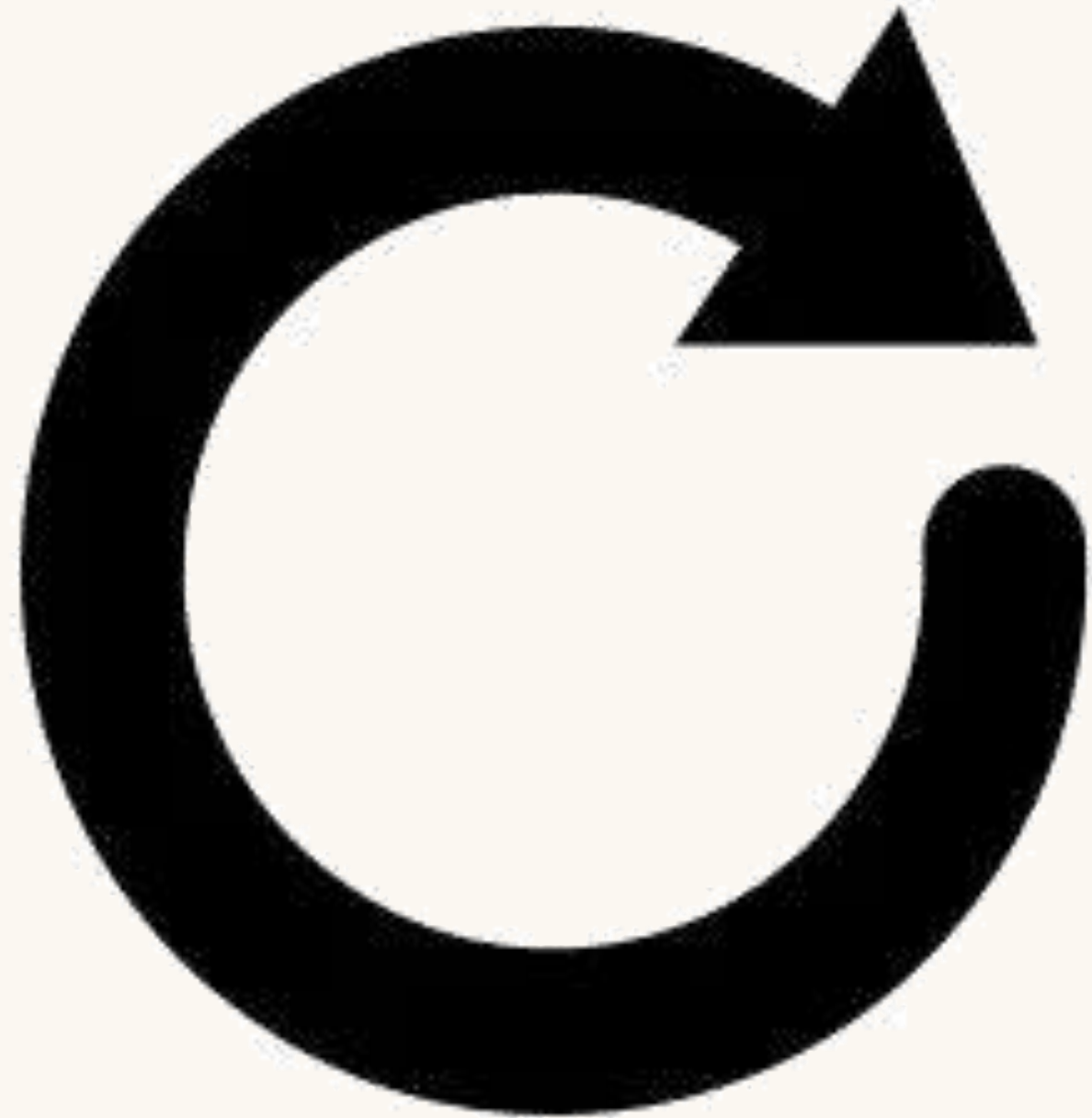
OTHERWISE:

OUTPUT "Invalid choice. Please try again."

ENDCASE

CASE Statement Example

```
DECLARE Grade : STRING
OUTPUT "Enter your grade (A, B, C, D, F): "
INPUT Grade
CASE Grade OF
    "A":
        OUTPUT "Excellent!"
    "B":
        OUTPUT "Very Good"
    "C":
        OUTPUT "Good"
    "D":
        OUTPUT "Pass"
    "F":
        OUTPUT "Fail"
    OTHERWISE
        OUTPUT "Invalid grade entered."
ENDCASE
```



LOOPS REPEAT
ACTIONS...

For Loop

```
FOR counter ← start TO end  
    // Statements to repeat  
NEXT counter
```

```
FOR i ← 1 TO 5  
    OUTPUT i  
NEXT i
```

```
DECLARE Sum : INTEGER  
DECLARE I : INTEGER  
Sum <- 0  
FOR I← 1 TO 10  
    sum ← sum + I  
NEXT I  
  
OUTPUT "Total Sum: ", sum
```

For Loop with STEP

```
DECLARE I : INTEGER  
FOR I ← 50 TO 1 STEP -5  
  OUTPUT I  
NEXT I
```

```
DECLARE I : INTEGER  
FOR I ← 1 TO 50 STEP 5  
  OUTPUT I  
NEXT I
```

Nested For Loop

```
FOR outer ← start1 TO end1
  FOR inner ← start2 TO end2
    // Statements using outer and inner
  NEXT inner
NEXT outer
```

```
DECLARE I : INTEGER
DECLARE J : INTEGER

FOR I ← 1 TO 5
  FOR J ← 1 TO 5
    OUTPUT I * J;
  NEXT J
  OUTPUT "" // Move to next line
NEXT I
```

While Loop

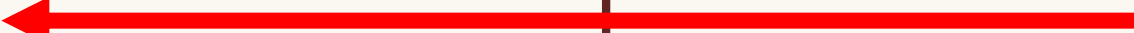
```
WHILE condition DO  
    // Statements to repeat  
ENDWHILE
```

The loop keeps running as long as condition is TRUE

Note: Be careful for **WHILE** Loop. Your program may stuck infinite loop

```
DECLARE I : INTEGER  
I ← 1  
WHILE I <= 5 DO  
    OUTPUT I  
    I ← I + 1  
ENDWHILE
```

Pre-Condition Loop: First Verify the condition then process the loop



While Loop

```
DECLARE Password : STRING
```

```
DECLARE Correct : STRING
```

```
Correct ← "open123"
```

```
Password ← ""
```

```
WHILE Password <> Correct DO
```

```
    OUTPUT "Enter password: "
```

```
    INPUT Password
```

```
ENDWHILE
```

```
OUTPUT "Access granted!"
```


Repeat

```
REPEAT
```

```
  // Statements to repeat
```

```
UNTIL condition
```

```
DECLARE I : INTEGER
```

```
I ← 0
```

```
REPEAT
```

```
  OUTPUT I
```

```
  I ← I + 1
```

```
UNTIL I > 5
```

Post-Condition Loop: First execute the loop (At least Once) then verify the condition



Repeat

```
DECLARE Num : INTEGER
```

```
DECLARE Sum : INTEGER
```

```
Num ← 0
```

```
Sum ← 0
```

```
REPEAT
```

```
    OUTPUT "Enter a number (0 to stop): "
```

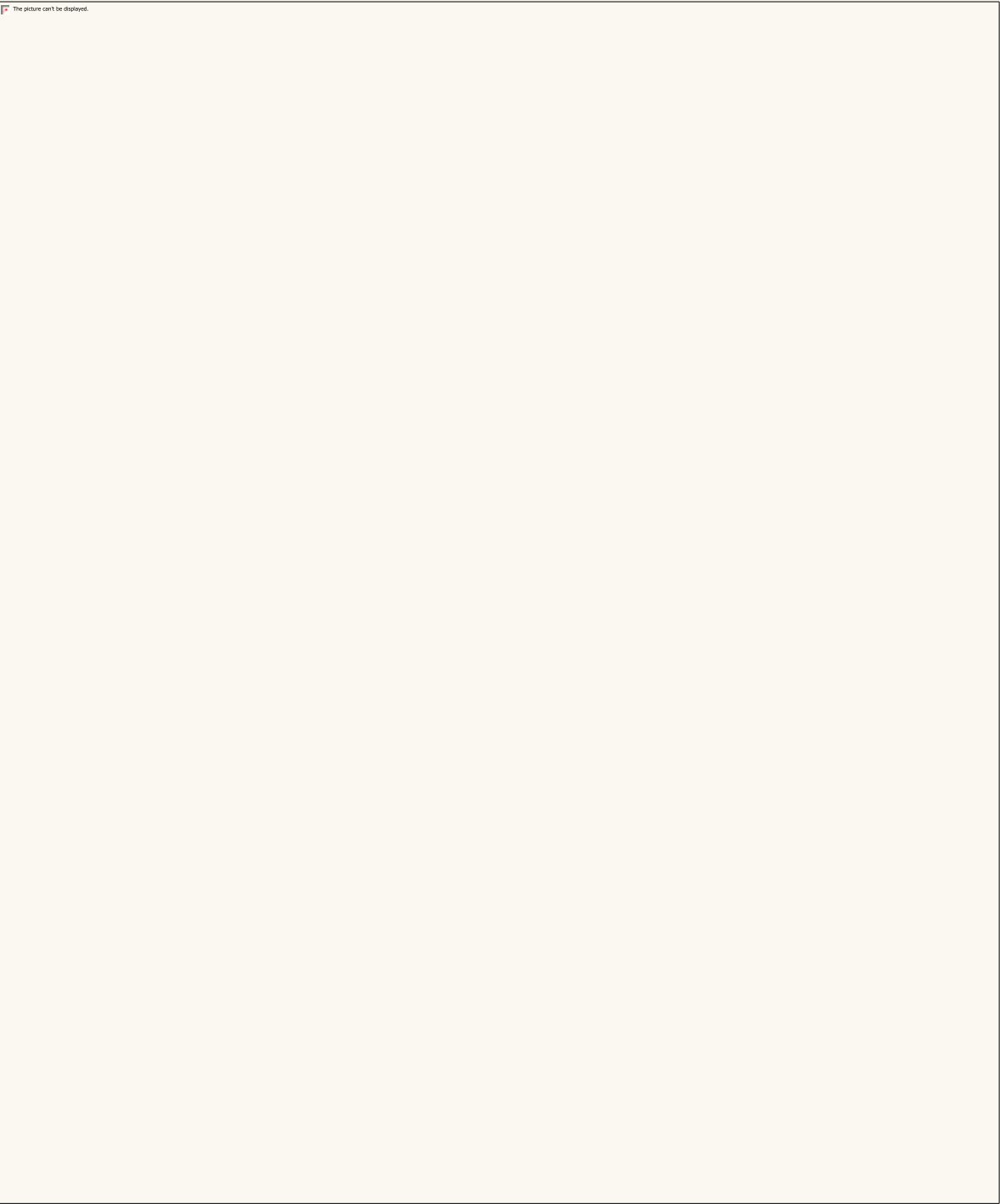
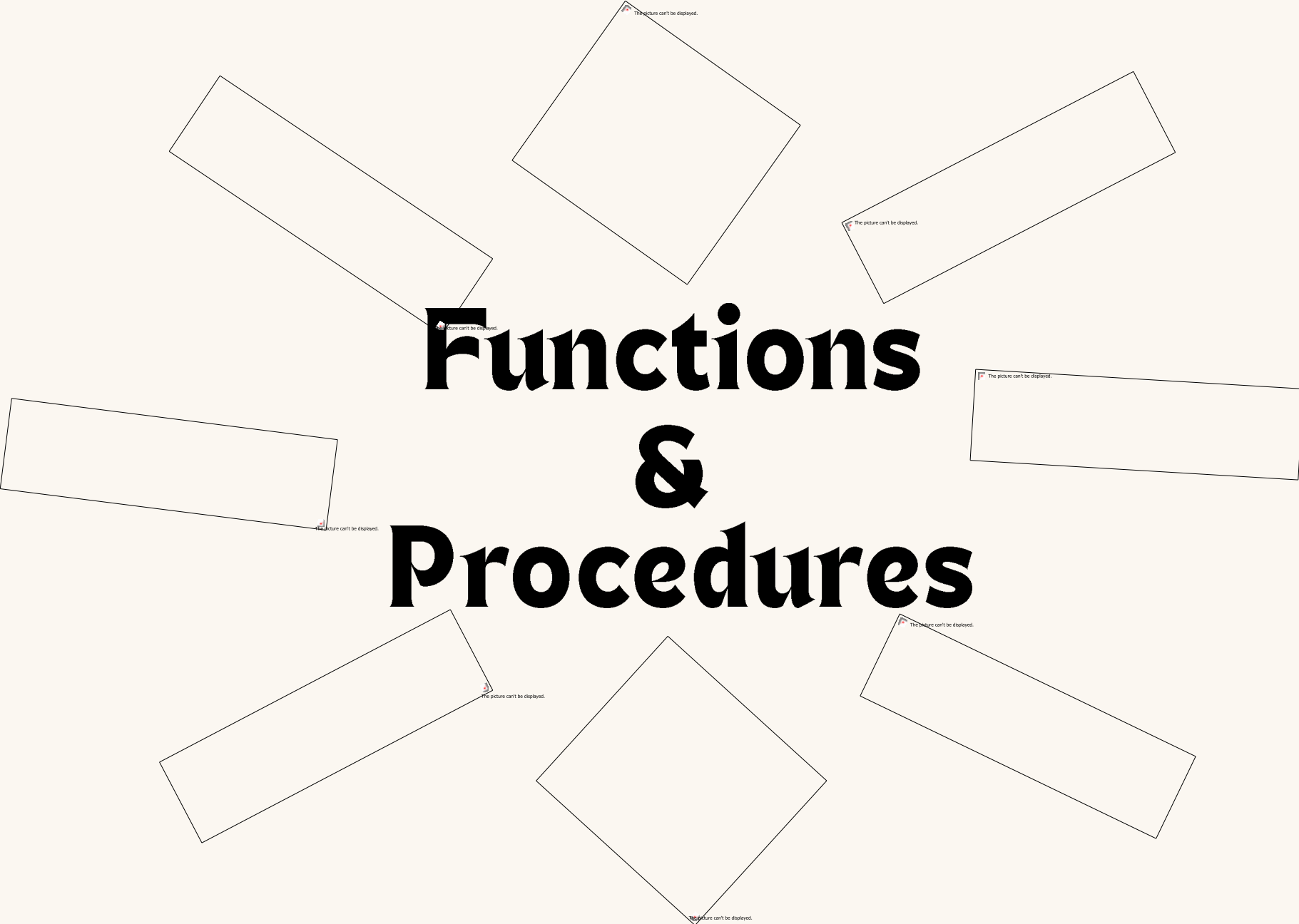
```
    INPUT Num
```

```
    Sum ← Sum + Num
```

```
UNTIL Num = 0
```

```
OUTPUT "Total sum is: ", Sum
```

Functions & Procedures



Functions & Procedures

Feature	Procedure	Function
Returns value?	✗ No	✓ Yes (returns a value)
Usage	Performs a task	Performs a task and gives a result
Called by	Name	Name and often inside a statement or expression

Functions & Procedures

```
FUNCTION FunctionName(parameter1:Data type, parameter2:Data Type) RETURNS Data Types
```

```
    // Code to calculate
```

```
    RETURN result
```

```
ENDFUNCTION
```

```
FUNCTION Add(X:INTEGER, Y:INTEGER) RETURNS INTEGER
```

```
    RETURN X + Y
```

```
ENDFUNCTION
```

```
// Using the function
```

```
DECLARE Total : INTEGER
```

```
Total <- Add(5, 7)
```

```
OUTPUT "Total: ", Total
```

<https://cs.coursemo.com/igpc/>
<https://pseudocode.pro>

Functions & Procedures

```
PROCEDURE ProcedureName(parameter1:Data type, parameter2:Data Type)
```

```
// Code to perform
```

```
ENDPROCEDURE
```

```
PROCEDURE Greet(Name : STRING)
```

```
    OUTPUT "Assalam O Alikum, " , Name , "!"
```

```
ENDPROCEDURE
```

```
// Calling the procedure
```

```
CALL Greet("Fatima")
```

Functions & Procedures (combined Example)

```
//*****  
// Function Declaration  
  
FUNCTION Square(Num : INTEGER) RETURNS INTEGER  
    RETURN Num * Num  
ENDFUNCTION  
//*****  
  
// Procedure Declaration  
//*****  
PROCEDURE DisplaySquare(Num : INTEGER)  
    DECLARE Result : INTEGER  
    Result <- Square(Num)  
    OUTPUT "Square of ", Num, " is ", Result  
ENDPROCEDURE  
//*****  
  
DECLARE InputNumber : INTEGER  
OUTPUT "Enter Your Number"  
INPUT InputNumber  
CALL DisplaySquare(InputNumber)
```

FILE HANDLING



File Handling

What is file handling?

- File handling is **the use of programming techniques** to work with information stored in **text files**
- Examples of file handing techniques are:
 - **opening** text files
 - **reading** text files
 - **writing** text files
 - **closing** text files

Concept	OCR exam reference	Python
Open	OPENFILE "fruit.txt" FOR READ	file = open("fruit.txt","r")
Close	CLOSEFILE "fruit.txt"	file.close()
Read line	READFILE "fruit.txt", LineOfText	file.readline()
Write line	OPENFILE "fruit.txt" FOR WRITE WRITEFILE "fruit.txt", "Oranges"	file.write("Oranges")
Append a file	OPENFILE "fruit.txt" FOR APPEND	file = open("shopping.txt","a")

File Handling (example)

```
DECLARE file : STRING
DECLARE line : STRING
file <-- "myfile.txt"
OPENFILE file FOR WRITE
WRITEFILE file, "Hello"
WRITEFILE file, "world"
CLOSEFILE file

//can also use filename directly, not as a string literal
OPENFILE myfile.txt FOR READ
READFILE myfile.txt, line
OUTPUT line
READFILE myfile.txt, line
OUTPUT line
CLOSEFILE myfile.txt

OUTPUT "Reading using EOF (end of file) function:"
OPENFILE file FOR READ
WHILE NOT EOF(file) DO
    READFILE file, line
    OUTPUT line
ENDWHILE
CLOSEFILE file
```

File Handling (example)

```
DECLARE file : STRING  
DECLARE line : STRING  
DECLARE InputString : STRING
```

```
file <-- "myfile.txt"
```

```
OPENFILE file FOR WRITE  
OUTPUT "Enter any Suitable String"  
INPUT InputString  
WRITEFILE file, InputString  
CLOSEFILE file
```

```
//can also use filename directly, not as a string literal  
OPENFILE myfile.txt FOR READ  
READFILE myfile.txt, line  
OUTPUT line  
CLOSEFILE myfile.txt
```

```
OUTPUT "Reading using EOF (end of file) function:"  
OPENFILE file FOR READ  
WHILE NOT EOF(file) DO  
    READFILE file, line  
    OUTPUT line  
ENDWHILE  
CLOSEFILE file
```


Library Routines - String Handling

- length** – finding the number of characters in the string.
- substring** – extracting part of a string.
- upper** – converting all the letters in a string to uppercase.
- lower** – converting all the letters in a string to lowercase.

Examples

LENGTH(string)	Returns number of characters	LENGTH("hello") → 5
SUBSTRING(str, x, y)	Returns substring from position x for y chars	SUBSTRING("hello", 2, 3) → "ell"
UPPERCASE(string)	Converts all letters to uppercase	UPPERCASE("hi") → "HI"
LOWERCASE(string)	Converts all letters to lowercase	LOWERCASE("Hi") → "hi"

String Handling (example)

```
DECLARE TextLine : STRING // variables are declared as normal
DECLARE MyFile : STRING
MyFile <-- "myfile.txt"
OPENFILE MyFile FOR WRITE
OUTPUT "Please enter a line of text"
INPUT TextLine
WRITEFILE MyFile,TextLine // writes a line of text to the file
CLOSEFILE(MyFile) // closes the file
// reading the line of text from the file

OUTPUT "The file contains this line of text:"
OPENFILE MyFile FOR READ
READFILE MyFile , TextLine

OUTPUT LENGTH (TextLine)
OUTPUT SUBSTRING (TextLine,1,4)
OUTPUT TO_UPPER (TextLine)
OUTPUT TO_LOWER(TextLine)

OUTPUT TextLine
CLOSEFILE(MyFile) // closes the file
```

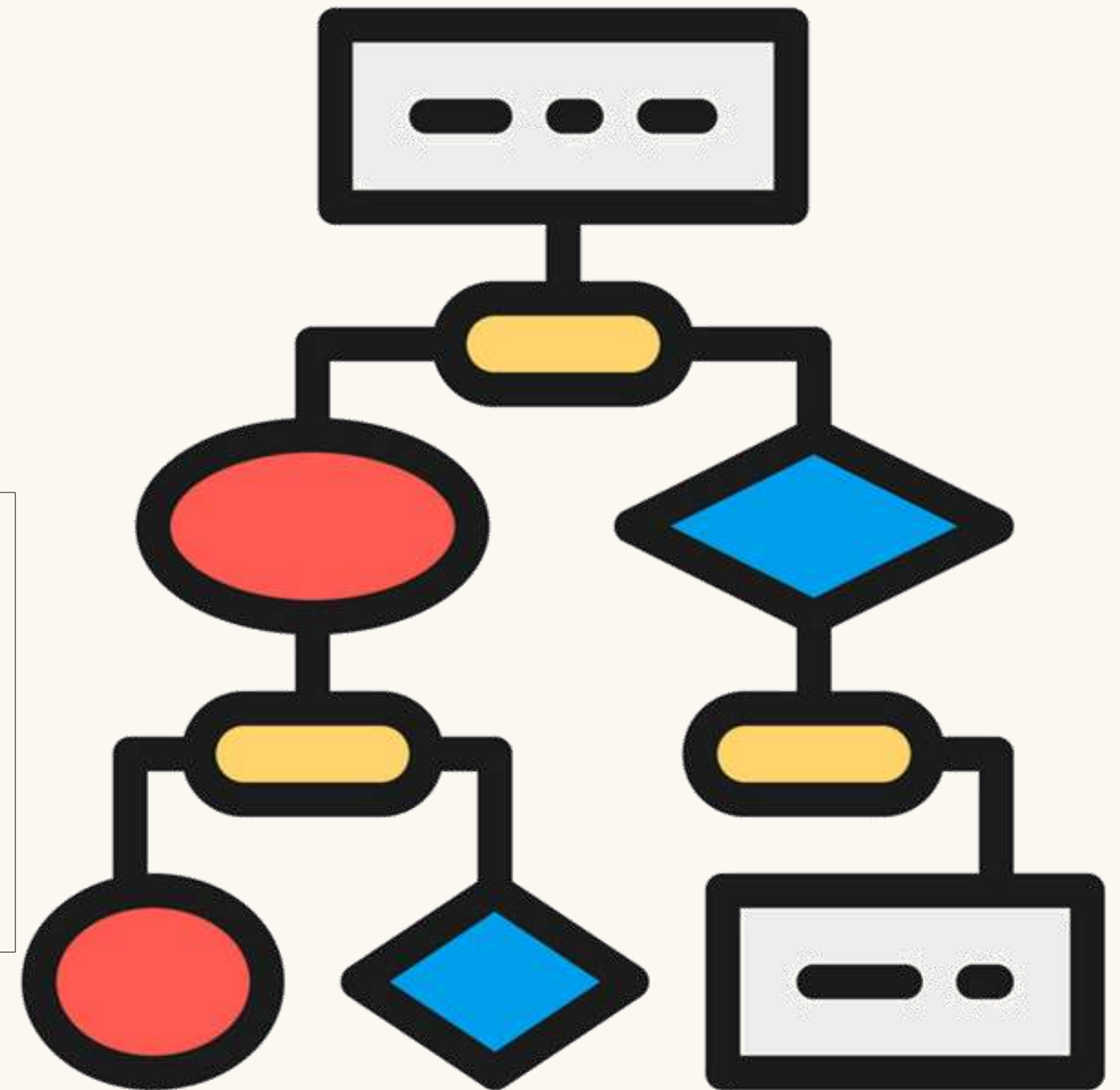
Library Routines - Math's

Examples

- MOD** – returns remainder of a division
- DIV** – returns the quotient (i.e. the whole number part) of a division
- ROUND** – returns a value rounded to a given number of decimal places
- RANDOM** – returns a random number.

MOD(14,5)	MOD(14, 5) = 4 - i.e. how many slices are left over, after everyone has been given their 2 whole slices
DIV(14,5)	DIV(14, 5) = 2 - i.e. how many whole slices each person gets
ROUND(2.7556,3)	Rounds a real number to a given number of decimal places Parameters: [REAL] - the number to round [INTEGER] - the number of decimal places to round it to
RANDOM(.5)	Returns a real number between 0-1 inclusive

Algorithms



Algorithms

- **Totaling**
- **Counting**
- **Finding maximum, minimum, and average (mean) values**
- **Searching using a linear search**
- **Sorting using a bubble sort.**

Totaling and Counting (Differences)

Concept	Totaling	Counting
Goal	Add up values	Count how many meet a rule
Output	A sum	A number of times something happens

Totaling

- **Totalling involves adding up values, often in a loop**
- **A total variable can be initialized to 0 and then updated within a loop**

```
total  $\leftarrow$  0  
FOR i  $\leftarrow$  1 TO N DO  
    total  $\leftarrow$  total + value[i]  
ENDFOR  
OUTPUT total
```

The picture can't be displayed.

Counting

- Counting involves keeping track of the number of times a particular event occurs
- A count variable can be initialized to 0 and then updated within a loop

```
count ← 0
FOR i ← 1 TO N DO
    IF value[i] > 10 THEN
        count ← count + 1
    ENDIF
ENDFOR
OUTPUT count
```


Finding **Max, Min**, Mean (Average) and Median

DECLARE Number_of_students, Mark, Highest, Lowest, i : INTEGER

INPUT Number_of_students

INPUT Mark

Highest $\leftarrow$ Mark

Lowest $\leftarrow$ Mark

FOR i $\leftarrow$ 2 TO Number_of_students

 INPUT Mark

 IF Mark > Highest THEN

 Highest $\leftarrow$ Mark

 ENDIF

 IF Mark < Lowest THEN

 Lowest $\leftarrow$ Mark

 ENDIF

NEXT i

OUTPUT "Highest Mark: ", Highest

OUTPUT "Lowest Mark: ", Lowest

Finding Max, Min, **Mean (Average)** and Median

DECLARE Number_of_students, Mark, Highest, Lowest, i , **Total_Sum**: INTEGER

INPUT Number_of_students

INPUT Mark

Highest $\leftarrow$ Mark

Lowest $\leftarrow$ Mark

Total_Sum $\leftarrow$ 0

FOR i $\leftarrow$ 2 TO Number_of_students

INPUT Mark

Total_Sum $\leftarrow$ Total_Sum + Mark

IF Mark > Highest THEN

Highest $\leftarrow$ Mark

ENDIF

IF Mark < Lowest THEN

Lowest $\leftarrow$ Mark

ENDIF

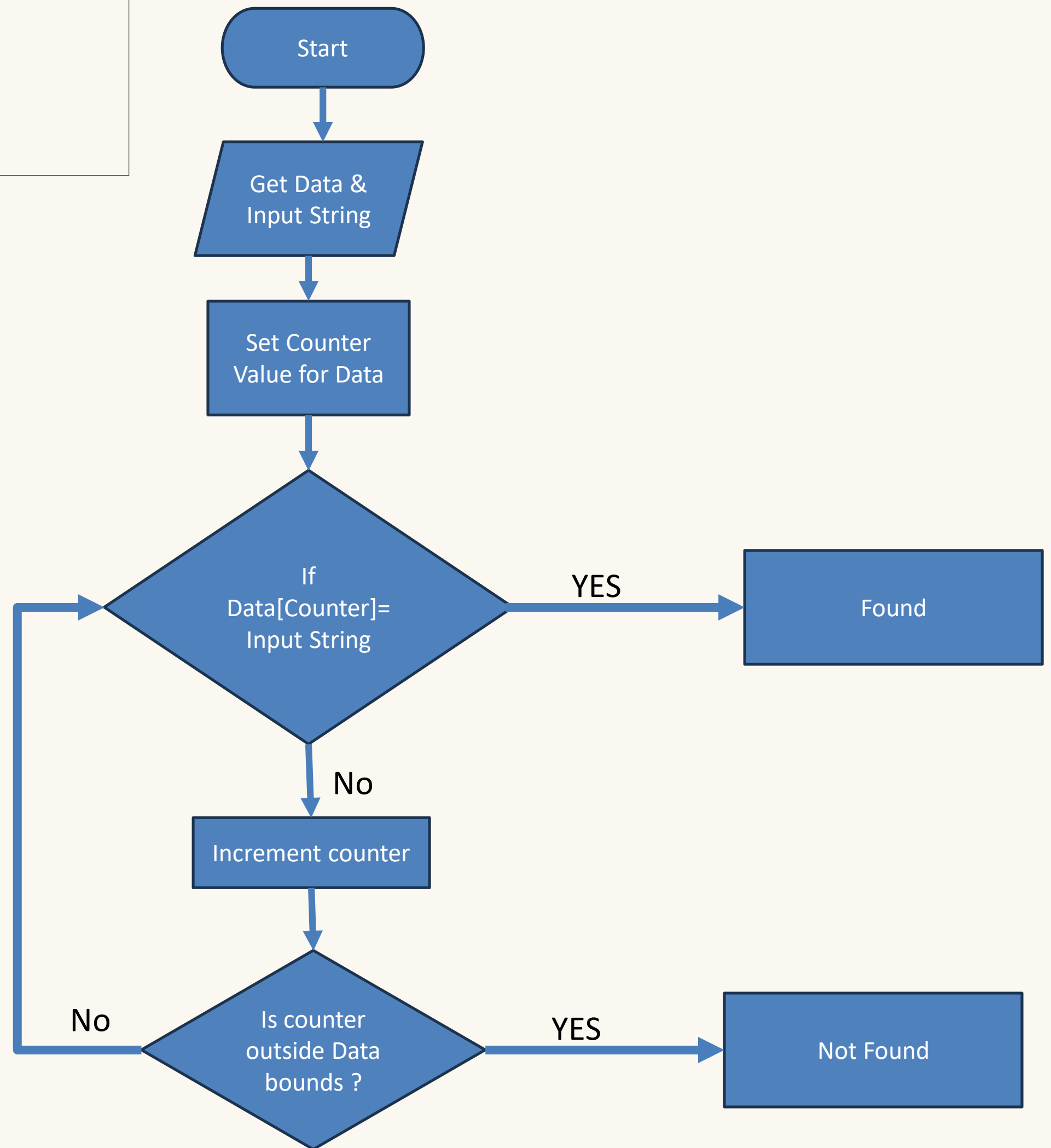
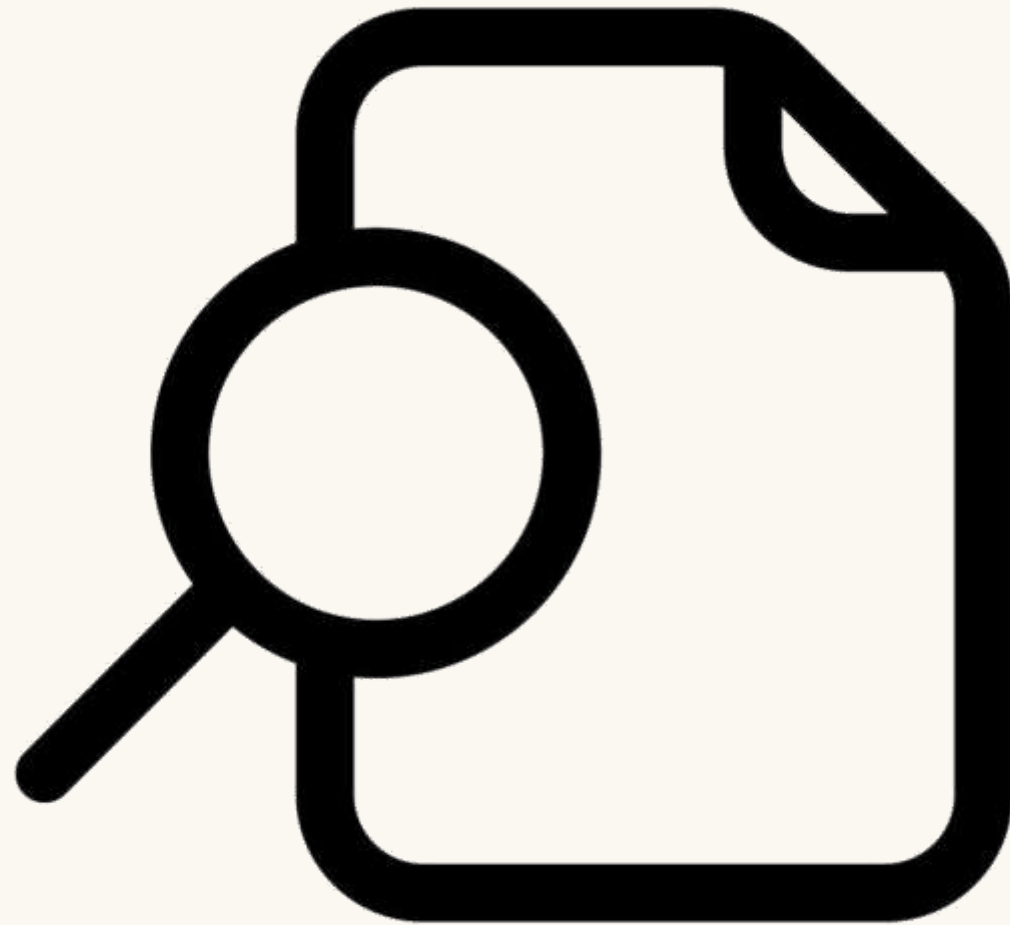
NEXT i

OUTPUT "Highest Mark: ", Highest

OUTPUT "Lowest Mark: ", Lowest

OUTPUT " Average Score : ", Total_Sum /Number_of_students

Linear Search



Linear Search

Start

1. Input: search_Item, number_Of_Elements
2. Input: the list elements (in a loop or represented together)
3. Initialize: found $\leftarrow$ false
4. $i \leftarrow 1$
5. Decision: $i \leq \text{number_Of_Elements}$?
6. Yes $\rightarrow$ next step
7. No $\rightarrow$ go to Output "Item not found", then End
8. Decision: $\text{list}[i] = \text{search_Item}$?
9. Yes $\rightarrow$ set found $\leftarrow$ true, store position, Output "Item found", then End
10. No $\rightarrow$ increment $i \leftarrow i + 1$, go back to step 5

End

Linear Search - Question

Declare Array of following names :
Alice, Bob, Charlie, Diana, Eve

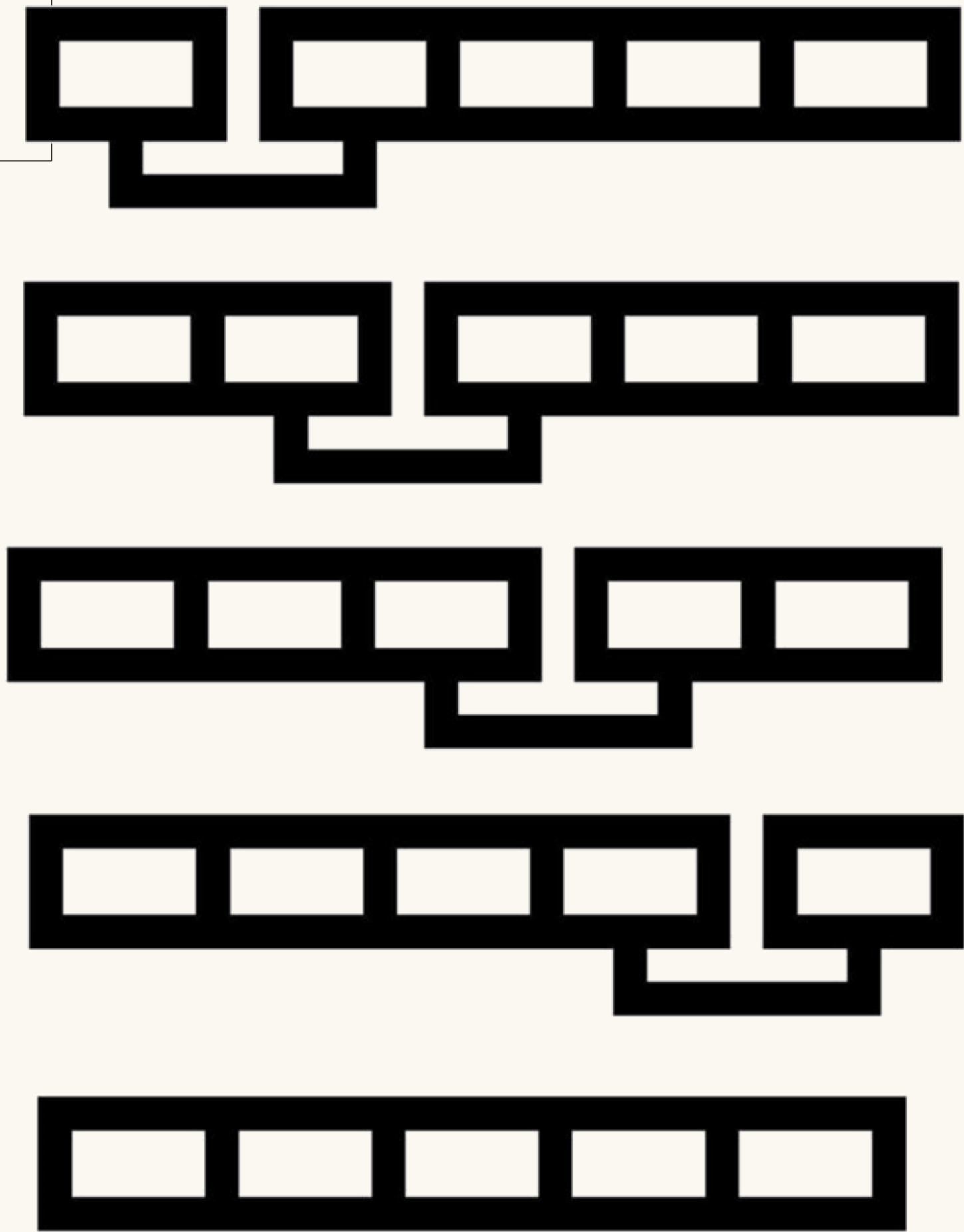
Ask input for name search

As a result, Name found or Not , if yes then at
which position in Array



Bubble Sort

Bubble Sort is a simple sorting algorithm that compares **adjacent elements** in a list and **swaps** them if they are in the wrong order. This process is repeated until the list is sorted



Bubble Sort

1. Set $N = \text{Length of Array}$

2. $\text{Array}[j] > \text{Array}[j + 1]$ (Decision)

// if no - continue with the step 4

3. Swap $\text{Array}[j]$ and $\text{Array}[j+1]$

4. Increment j ($j++$)

5. $j < N - i - 1$ (Decision)

// if yes - continue with the step 2

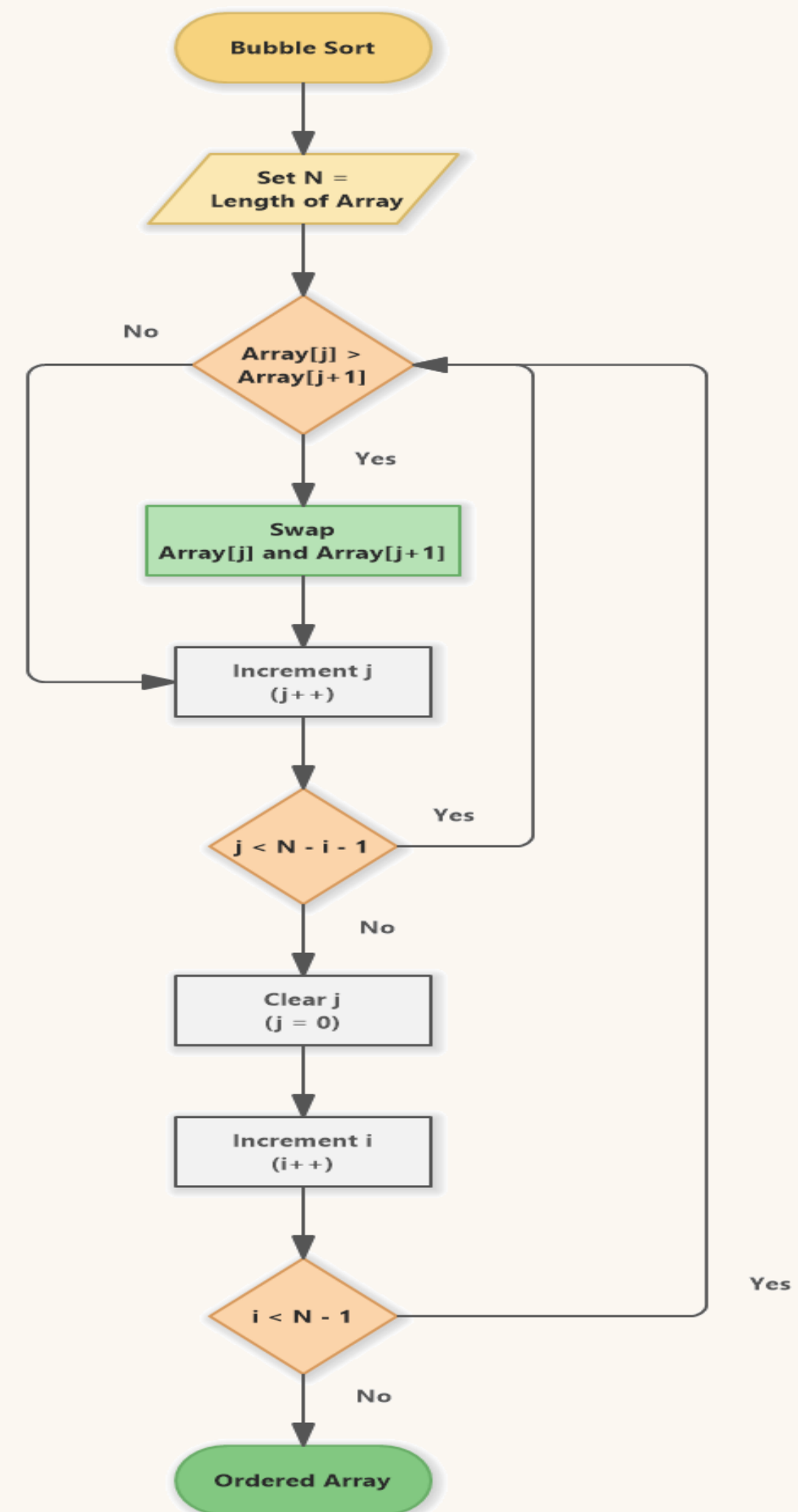
6. Clear j ($j = 0$)

7. Increment i ($i++$)

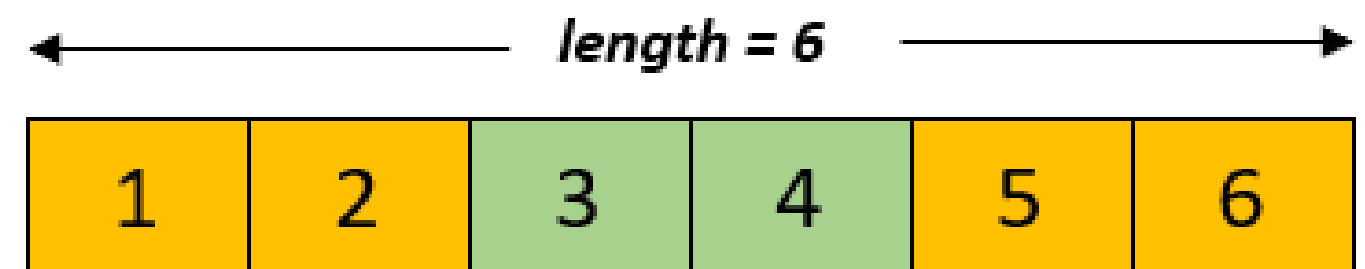
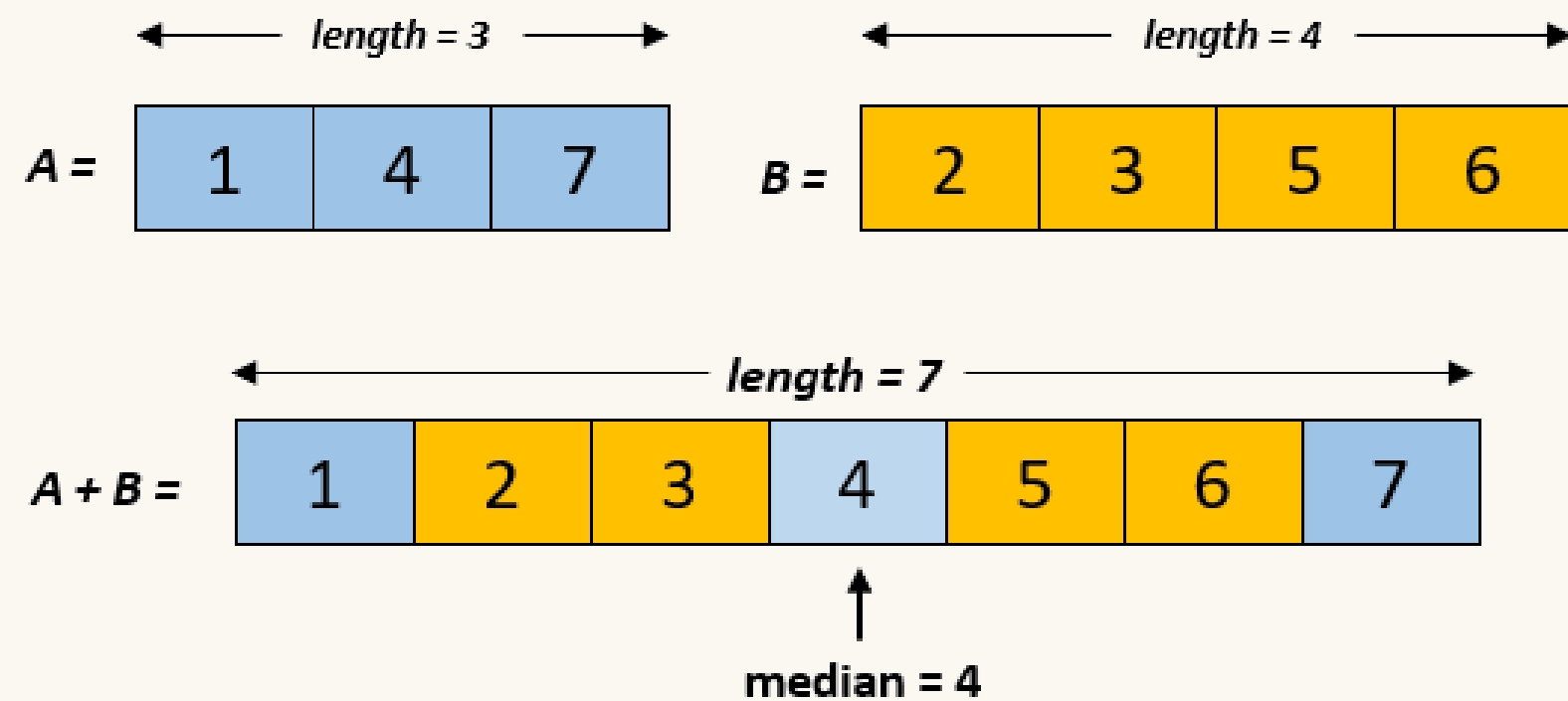
8. $i < N - 1$ (Decision)

// if yes continue with the step 2

9. Ordered Array



Finding Median



Finding Median

05 Students are in a class

Arrange their marks in ascending order and find the median in marks

Display median marks and corresponding Student Name

If No Students are even category display both median Marks

Student Name	Marks Case 1
Asim	56
Baber	85
Dawood	54
Zubair	32
Talha	26

Student Name	Marks Case 2
Asim	56
Baber	85
Dawood	54
Zubair	32
Talha	26
Qasim	56

Validation Categories

Validation is an **automated process** where a **computer checks** if a user input is **sensible** and meets the program's requirements.

- **RANGE CHECKS**
- **LENGTH CHECKS**
- **TYPE CHECKS**
- **PRESENCE CHECKS**
- **FORMAT CHECKS**
- **CHECK DIGIT**

Range Check

Range check

- Ensures the data entered as a number **falls within a particular range**

Pseudocode	Ensures a percentage number is between 0-100 inclusive
DECLARE Number : INTEGER	
OUTPUT "Enter a number between 0 and 100"	
REPEAT	
INPUT Number	
IF Number < 0 OR Number > 100	
THEN	
OUTPUT "Number is not between 0 and 100, please try again"	
ENDIF	
UNTIL Number >= 0 AND Number <= 100	

Length Check

Checks the **length of a string**

Pseudocode	
DECLARE Pin : STRING	
OUTPUT "Please enter your 4 digit bank PIN number"	
REPEAT	
INPUT Pin	
IF LENGTH(Pin) <> 4	
THEN	
OUTPUT "Your pin number must be four characters in length, please try again"	
ENDIF	
UNTIL LENGTH(Pin) = 4	

Type Check

Check the **data type** of a field

Pseudocode	Ensures an age should be entered as an integer (whole number)
DECLARE Number : INTEGER	
OUTPUT "Enter an integer number"	
REPEAT	
INPUT Number	
IF Number <> DIV(Number, 1)	
THEN	
OUTPUT "Not a whole number, please try again"	
ENDIF	
UNTIL Number = DIV(Number , 1)	

Presence Check

Looks to see if **any data has been entered** in a field

Pseudocode	Ensures user name and password are both entered
<pre>DECLARE Username : STRING OUTPUT "Enter your username" REPEAT INPUT Username IF Username = "" THEN OUTPUT "No username entered, please try again" ENDIF UNTIL Username <> ""</pre>	

Format Check

- Ensures that the data has been entered in the **correct format**
- Format checks are done using **pattern matching** and string handling

<pre>INPUT IDNumber IF LENGTH(IDNumber) <> 6 THEN OUTPUT "ID number must be 6 characters long" END IF ValidChars ← "ABCDEFGHIJKLMNOPQRSTUVWXYZ" FirstChar ← SUBSTRING(IDNumber, 1, 1) ValidChar ← False Index ← 1 WHILE Index <= LENGTH(ValidChars) AND ValidChar = False DO IF FirstChar = ValidChars[Index] THEN ValidChar ← True ENDIF Index ← Index + 1 ENDWHILE IF ValidChar = False THEN OUTPUT "First character is not a valid uppercase alphabetical character" ENDIF SecondChar ← SUBSTRING(IDNumber, 2, 2) ValidChar ← False Index ← 1 WHILE Index <= LENGTH(ValidChars) AND ValidChar = False DO</pre>	<pre>IF SecondChar = ValidChars[Index] THEN ValidChar ← True ENDIF Index ← Index + 1 ENDWHILE IF ValidChar = False THEN OUTPUT "Second character is not a valid uppercase alphabetical character" ENDIF Digits ← INT(SUBSTRING(IDNumber, 3, 6)) IF Digits < 0000 OR Digits > 9999 THEN OUTPUT "Digits invalid. Enter four valid digits in the range 0000-9999" ENDIF</pre>
---	--

Check Digit

- Check digits are **numerical values** that are the **final digit of a larger code** such as a **barcode** or an International Standard Book Number (**ISBN**)
- They are calculated by **applying an algorithm** to the code and are then attached to the overall code

Check Digit = (Sum of digits) MOD 10

EXAMPLE:

Input: "3725"

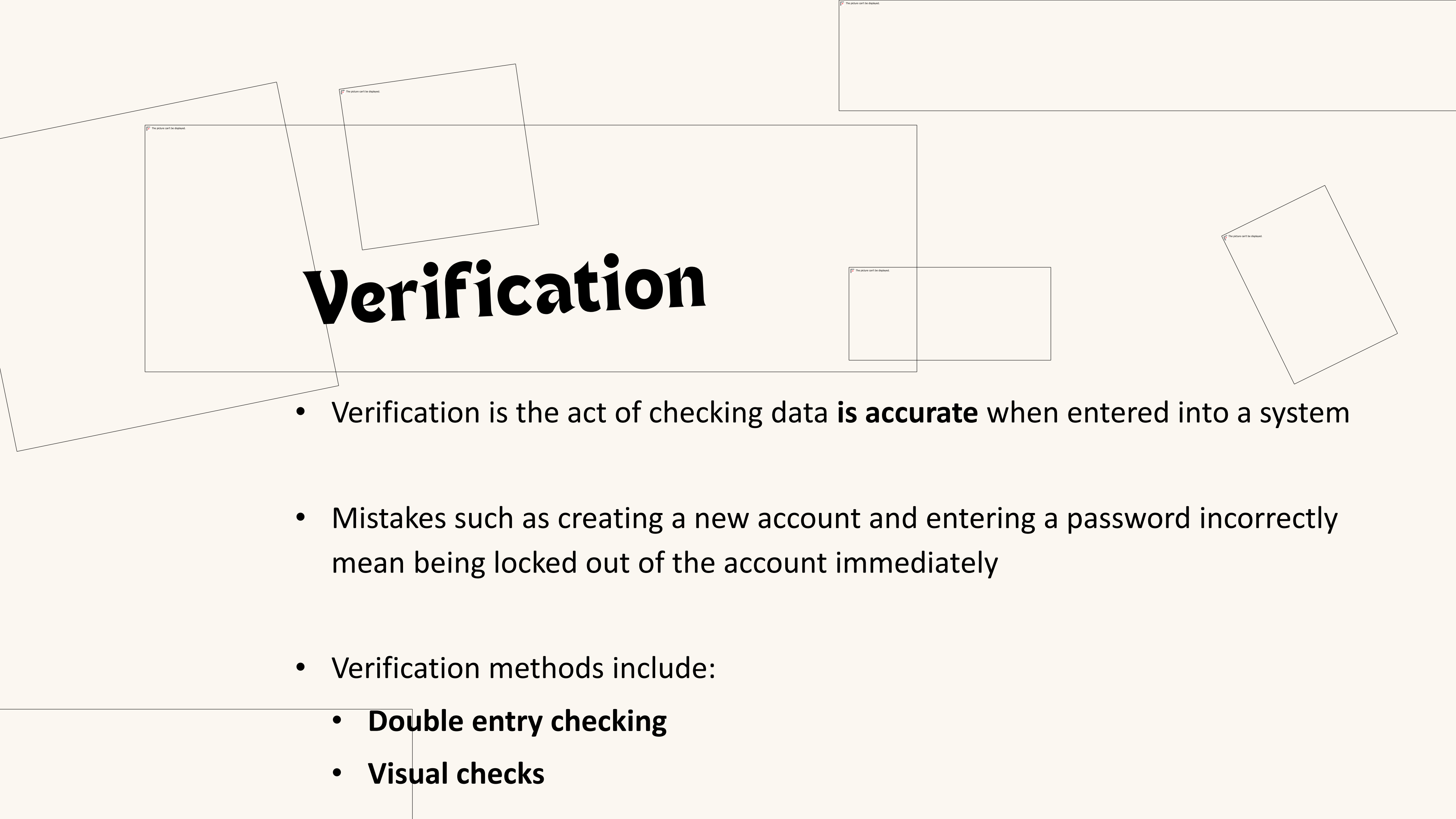
Sum = 3 + 7 + 2 + 5 = 17

17 MOD 10 = **7** → check digit is **7**

Full number: 37257

```
PROCEDURE CalculateCheckDigit(numberString)
    total ← 0
    FOR i ← 0 TO LENGTH(numberString) - 1
        digit ← STRING_TO_INT(numberString[i])
        total ← total + digit
    NEXT i
    checkDigit ← total MOD 10
    RETURN checkDigit
ENDPROCEDURE
```

Implement Code using Shown procedure and given example



Verification

- Verification is the act of checking data **is accurate** when entered into a system
- Mistakes such as creating a new account and entering a password incorrectly mean being locked out of the account immediately
- Verification methods include:
 - **Double entry checking**
 - **Visual checks**

Double Entry Checking

- Double entry checking involves **entering the data twice** in separate input boxes and then comparing the data to ensure they both match
- If they do not, an error message is shown

```
DECLARE Password, ConfirmPassword : STRING
```

```
REPEAT
```

```
    OUTPUT "Enter your password"
```

```
    INPUT Password
```

```
    OUTPUT "Please confirm your password"
```

```
    INPUT ConfirmPassword
```

```
    IF Password <> ConfirmPassword
```

```
        THEN
```

```
            OUTPUT "Passwords do not match, please try again"
```

```
        ENDIF
```

```
UNTIL Password = ConfirmPassword
```

```
    OUTPUT "Passwords match"
```

Visual Check

- Visual checks involve the user **visually checking the data on the screen**
- A popup or message then asks if the data is correct before proceeding
- If it isn't the user then enters the data again

```
DECLARE Name, Answer: STRING
```

```
REPEAT
```

```
    OUTPUT "Enter your name"
```

```
    INPUT Name
```

```
    OUTPUT "Your name is: ", Name, ". Is this correct? (y/n)"
```

```
    INPUT Answer
```

```
UNTIL Answer = "y"
```

Commenting

Tip

- ✓ Start with a comment that explains the program
- ✓ Comment each major section
- ✓ Keep comments short and meaningful
- ✓ Explain tricky logic
- ✗ Don't restate the code in plain English unnecessarily

Explanation

- Describe what the program is meant to do
 - For loops, conditionals, input/output
 - Don't over-explain basic things
- If there's anything unusual or complex, explain it
 - Comments should add clarity, not repeat

Situation

Should You Comment?

Example

Explaining purpose of the program



// This program calculates class average

Inside a loop



// Repeating for each student

Obvious line like OUTPUT name



Not needed unless context is complex

Complex conditions



// Check if user is eligible AND verified

THANK
YOU!

